

COMP90015 Distributed Systems

Assignment 1 Multi-threaded Dictionary Server

Name: Shiqiang REN

Student ID: 1348968

Table of Contents

1 PROBLEMS AND REQUIREMENTS	3
2 THE COMPONENTS OF THE SYSTEM DESIGN.....	3
2.1 ARCHITECTURE AND COMMUNICATION PROTOCOLS	3
2.2 PARALLEL DESIGN.....	3
2.3 DATA SAFETY AND FORMAT	4
3 CLASS DESIGN AND INTERACTION DIAGRAM.....	4
3.1 CLIENT CLASS DESIGN	4
3.2 SERVER CLASS DESIGN	5
3.3 INTERACTION DIAGRAM	5
4 ANALYSIS	6
5 EXCELLENCE	7
5.1 USER INTERACTION.....	7
5.2 ANALYSIS OF THE SYSTEM IMPLEMENTED	7
6 CREATIVITY	7
6.1 SELF-IMPLEMENTED THREAD POOL	7
6.2 SERVER GUI FOR MANAGING	8

1 Problems and requirements

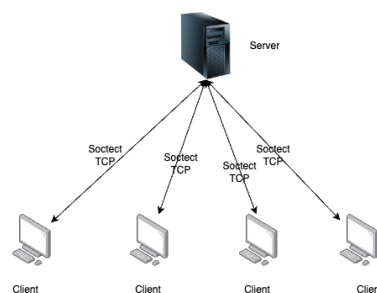
The task requires developing and implementing a dictionary query system that allows users to interact with a client by connecting to a server and sending requests such as querying, adding, updating, and deleting words to retrieve information and display it to the user.

On the server side, the system should be able to handle client requests in parallel and ensure the security and reliability of the data provided to clients. In terms of interaction, there should be a suitable GUI interface, appropriate error messages, etc. to improve the usability of the program

2 The components of the system design.

For the above problem and development requirements, this project has provided the following design and technology choices.

2.1 Architecture and communication protocols



The project adopts a client-server distributed architecture, where a single server can handle requests from different clients. The network connection between clients and servers uses Java sockets. Considering that the system transfers dictionary data, it is necessary to ensure the reliability of data transmission. Therefore, the project uses TCP connection. TCP protocol can monitor the transmission status during data transmission and automatically handle issues such as loss, damage, repetition, and order to ensure the reliability and integrity of data.

2.2 Parallel design

Firstly, both the client and server adopt the strategy of extracting UI loading as the main thread, and data processing as the sub-thread. This approach can improve user experience. For example, if there are time-consuming tasks such as data processing and network connection, they will not block UI rendering and interaction with users. This approach can also reduce coupling between different system functionalities, improve stability, and scalability.

For the server, the project adopts a thread-per-connection based work pool architecture. When a client sends a connection request, a waiting worker thread in the work pool will be taken out or a new worker thread will be created to handle the request. When the client closes the connection, the thread will be returned to the thread pool to wait for the next

connection. When multiple clients connect, the thread pool will take out multiple workers to process client requests in parallel. This design can improve the system's ability to handle many concurrent requests and improve performance by reusing threads.

2.3 Data safety and format

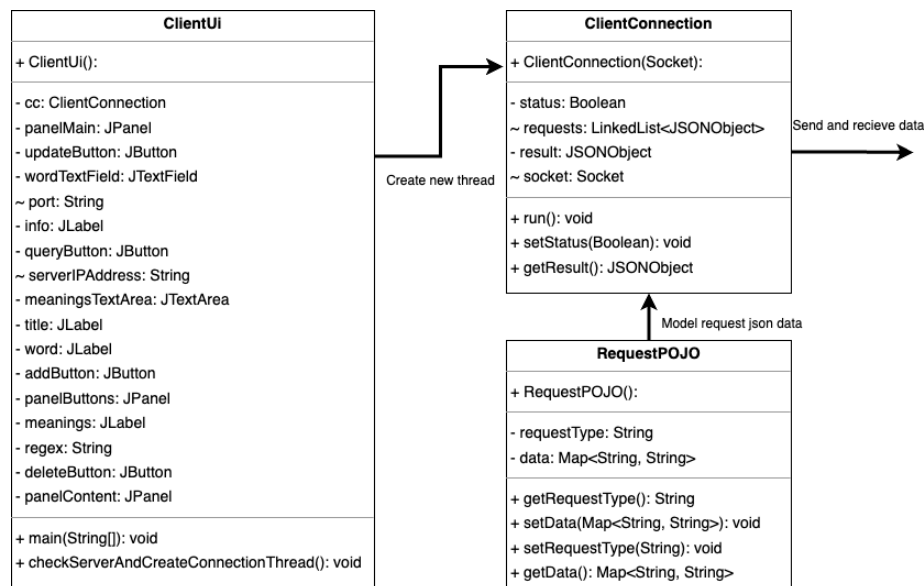
Due to the use of multi-threaded parallelism, data safety issues need to be considered for data shared by multiple threads. Two approaches have been adopted in this regard. For shared data between UI and connection threads, the synchronized keyword is used to lock thread-safe operations and ensure safety.

For the dictionary data on the server side, after the data is loaded into the memory, there may be many threads operating on it. Considering performance, the dictionary file data type is set as the ConcurrentHashMap, which has already implemented thread safety and performs better in operating elements.

The interaction between the client and the server, both message and dictionary data are formatted in JSON format. This is because the format is easy to use, has good performance, high readability, and strong extensibility.

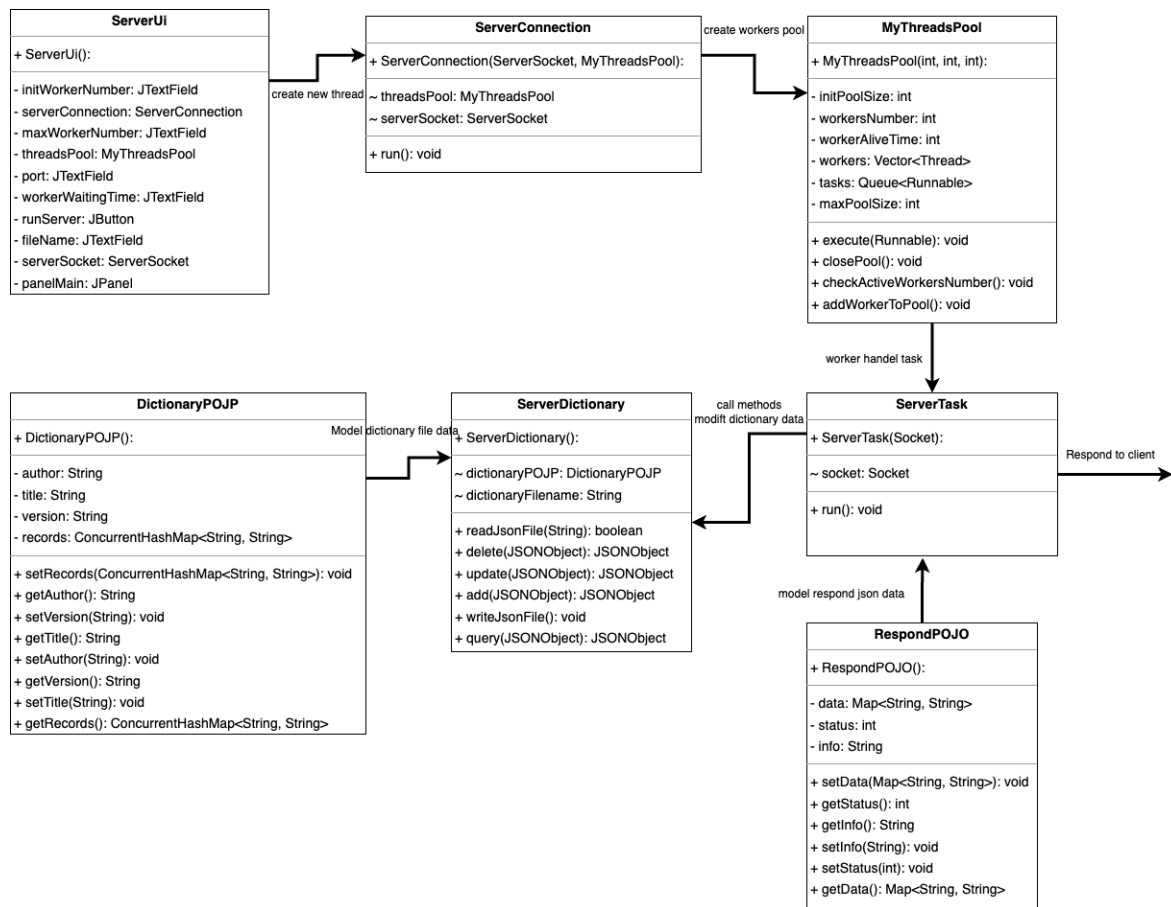
3 Class design and interaction diagram

3.1 Client class design



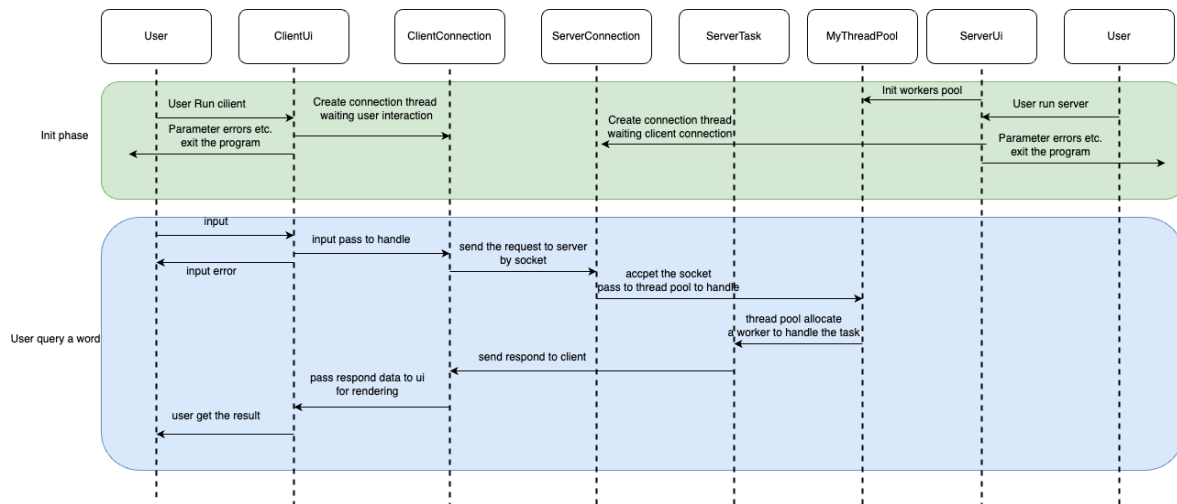
The ClientUI class is the main thread and entry point for loading the UI, checking, and connecting the server, etc. The ClientConnection class is a sub-thread for sending user requests and receiving server responses. RequestPOJO class used for modelling the request data.

3.2 Server class design



The **ServerUI** class is the main thread and entry point for loading the UI, checking the server setting parameters and creating thread pool, etc. The **ServerConnection** class is a sub-thread for and waiting the client socket connection. **ServerTask** class is for handling the client request task by calling **ServerDictionary** class methods like add, update, delete and query. **ServerDictionary** class also is for loading dictionary file data to memory and write the memory data to the disk file. **RespondPOJO** class and **DictionaryPOJP** used for modelling the respond data and dictionary data to save in the disk.

3.3 Interaction diagram



The interaction diagram above shows the progress of a client user send a query request and receive server respond data. The user input a word and click the query button, when the input is validated, the input will pass the sub-thread and send the request to the server by IO.

The serve accepts the client socket and builds the connection by a worker in the thread pool and waiting the request data. After getting the data, it will be handled by the relevant class methods and send the respond data back to the client. The worker will keep waiting the client request data.

4 Analysis

The project is a distributed system for dictionary queries, which implements client-server architecture. This architecture is high scalability and reliability, for example, it can be expanded into multiple servers to form a cluster, and if a server encounters an error, it ensures that the entire system can continue to run smoothly. In addition, Java sockets based on the TCP protocol are used for communication between clients and servers. As it is a dictionary query service, reliable information transmission needs to be ensured. The TCP protocol can achieve highly reliable data transmission through acknowledgment, retransmission, and sequencing. Finally, the project implements a self-implemented and scalable worker pool to handle client requests. Using a worker pool architecture can reuse threads, avoiding performance issues caused by frequent thread creation and destruction. This allows the system to handle many concurrent requests steadily. By creating thread pool by setting parameters (initial threads, maximum threads, thread wait time), The system can be better managed, avoiding underperformance problems and therefore improving system stability.

On the other hand, the system also has some drawbacks. The architecture of the client and server sometimes highly depends on the network situation, especially when dealing with large amounts of data and concurrent requests, network bandwidth and processing power can become system bottlenecks. The TCP protocol also has some issues, such as long connection times caused by the three-way handshake and low efficiency. The project's

thread pool also has certain limitations, such as when a client connects the server, the worker will be occupied until the user disconnected, this causes wastes server resources to some extent.

In summary, the project implements a client-server architecture based on TCP connections, using a scalable thread pool on the server to handle client requests for dictionary queries. It has a high degree of scalability and reliable information transmission. Additionally, using a scalable worker pool architecture can avoid performance and stability issues. However, the system also faces issues such as network bottlenecks and efficiency problems. In the future, the project can continue to add features such as when not active worker to handle the request when client connection timeout rejection and implementing server cluster.

5 Excellence

5.1 User interaction

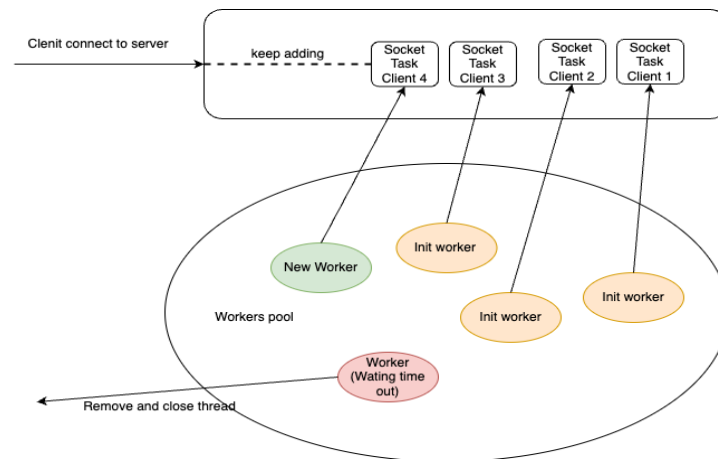
- The client user will be informed by a pop-up window when successfully connecting to the server and obtaining server data, such as successful deleting, updating, adding, word does not exist, word already exists, etc.
- The client user will also be informed by a pop-up window when encountering some errors, such as the server is not on, incorrect port number, incorrect server address.
- The client has added a pop-up window to verify the word input file.
- The client has added a pop-up window to confirm program exit.

5.2 Analysis of the system implemented

Section 4

6 Creativity

6.1 Self-implemented thread pool



Self-implemented a scalable worker pool that can handle client connections and data requests. When the server starts running, worker pool initializes a certain number of workers to wait for client connections based on the parameter settings. When a client socket is received, the worker in the thread pool retrieves the socket task and calls the relevant class's methods for processing. After processed and client disconnect from the server, it will put back to the thread pool and continues to wait for other sockets.

When all initialized workers in the thread pool are in working status, new threads are created to handle client sockets. When the task is completed and there are no client sockets to handle, the worker enters a waiting state. If the waiting time exceeds, the worker will be closed. When the number of workers equals the minimum number of workers initially set, the threads will not be closed and will remain in a waiting state.

6.2 Server GUI for managing

Implemented a GUI interface for a server to configure its starting parameters. Also implemented basic validation for the input parameters, such as checking for empty fields, verifying the existence of dictionary files, and ensuring that the thread pool size is appropriate.

